Deep Learning - lecture 4

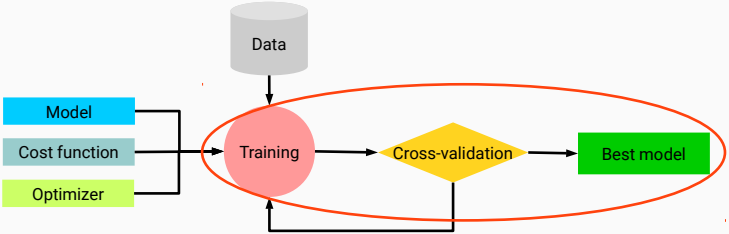
Model selection and hyperparameter tune

Prof. Stefano Carrazza

University of Milan and INFN Milan

Introduction

Outline


Data
Model
Cost function
Optimizer
Training
Cross-validation
Best model

Hyperparameters summary

So far we have encountered the following problems:

- Model:
  - model architecture / size
  - if NN: layers, nodes, activation functions
  - regularization techniques including early stopping techniques, weight decay, etc.
- Training:
  - performance metrics
  - optimizer configuration, e.g.: η, scheme, etc.
  - cross-validation split fractions
- Dataset:
  - size (gather more data?)
  - unbalanced data

Hyperparameters summary

So far we have encountered the following problems:

- Model:
  - model architecture / size
  - if NN: layers, nodes, activation functions
  - regularization techniques including early stopping techniques, weight decay, etc.
- Training:
  - performance metrics
  - optimizer configuration, e.g.: η, scheme, etc.
  - cross-validation split fractions
- Dataset:
  - size (gather more data?)
  - unbalanced data

Each choice should be tested $\rightarrow$ large space $\rightarrow$ difficult / time consuming.

Practical methodology

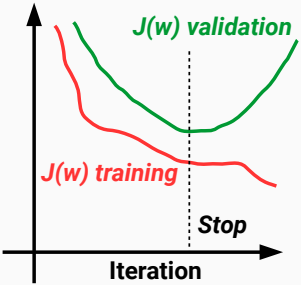
Don't get lost!

Designing a practical pipeline process:

- Estimate current state-of-the-art performance.
- Define realistic project goals, simplify / accelerate algorithms.
- Propose initial performance metrics matching the project goals.
- Perform incremental changes iteratively (data, hyperparameter, algorithms, etc.).

Example 1: auto-tuning model's capacity

How to simplify model capacity selection? Early stopping techniques



Monitor the cost function for the validation set and stop when it stops improving:

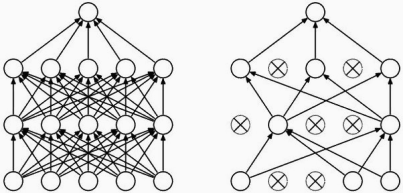
- look at the variation in a moving window
- stop at the minimum of the validation set (lookback method),

Example 2: auto-tuning model's capacity

How to simplify model capacity selection? Neural Network Dropout

At each training stage:

- individual nodes and related incoming and outgoing edges are dropped-out of the neural network with a fixed probability.
- the reduced NN is trained on the data.
- the removed nodes are reinserted in the NN with their original weights.

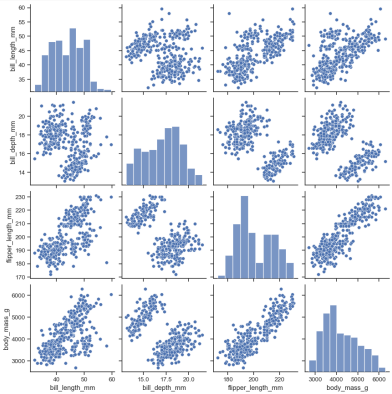


(a) Standard Neural Net (b) After applying dropout.

Data considerations

Data considerations

Step 1: Understand your data, extract correlations, perform minimal feature extraction.



Data considerations

Gathering more data is usually crucial but before:

Step 2:

- check the performance with the current training set, if its performance is:
  - **poor** $\rightarrow$ increase model size and tune the optimizer.
  - **acceptable** $\rightarrow$ check test set performance.

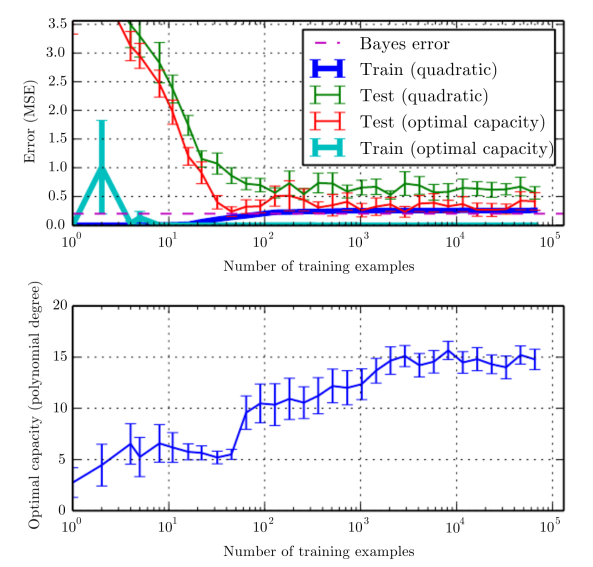
Step 3:

- if tuned models **fail** $\rightarrow$ check data for **noise or inconsistencies, collect new data**
- if test set performance is **poor** $\rightarrow$ **gather more data** if possible
  - if not possible **reduce the size of the model** or **improve hyperparameter tuning**.

Step 4:

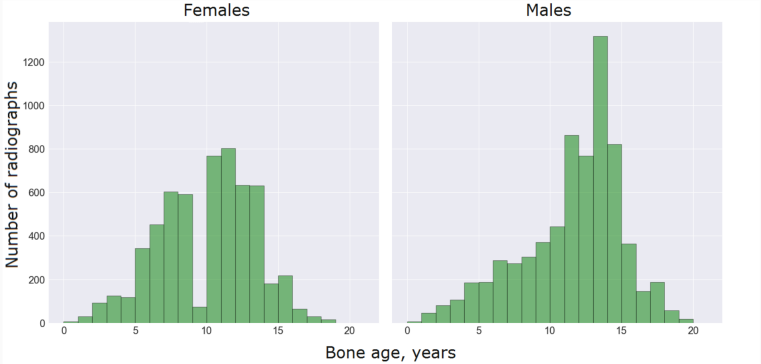
- estimate how much **additional training data** is needed.
- if gathering much more data is **not feasible** $\rightarrow$ improve the learning algorithm itself.

Data considerations



Unbalanced data

Unbalanced datasets are common issues:



Solution → perform **class weighting, oversampling, data augmentation**.

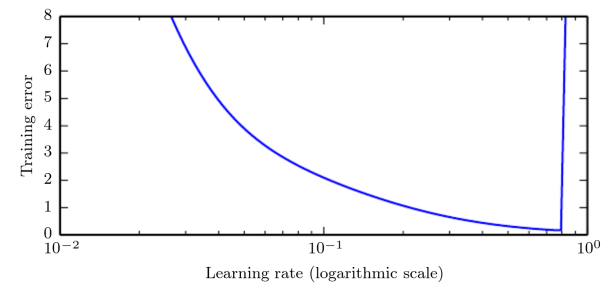
Hyperparameter tune

Manual hyperparameter tuning

Manual approach goal → achieve **good performance on the test set**.

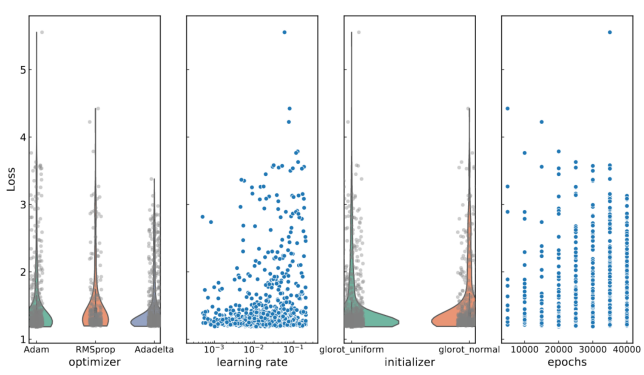
Some examples of effect of hyperparamaters on model capacity:

- Number of layers/nodes → increases capacity when increased.
- Learning rate → increases capacity when tuned optimally.
- Weight decay → increases capacity when decreased.
- Dropout rate → increases capacity when decreased.



Automatic hyperparameter optimization algorithms

Hyperparameter tuning is an **optimization problem** thus we can **automate the process**.



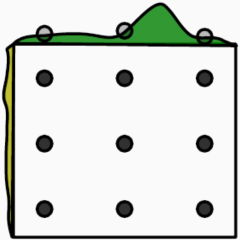
Common approaches:

- grid search
- random search
- bayesian optimization
- gradient-based optimization
- evolutionary optimization

Grid search

Grid search: searching through a manually subset range of the hyperparameter space.

- Train model for every grid point of the hyperparameter space.
- Allocate initial grids following a logarithmic scale, perform zoom in another round.
- Monitor the best validation set error $\rightarrow$ best hyperparameter values.



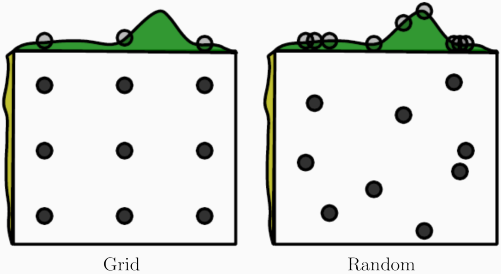
Grid

The disadvantage: with n values and m parameters the number of trials is $\mathcal{O}(n^m)$

Random search

Random search: sample trial points from a marginal distribution for each hyperparameter.

- Do not discretize or bin the values of the hyperparameters.
- The marginal distribution will perform independent explorations of hyperparameters.

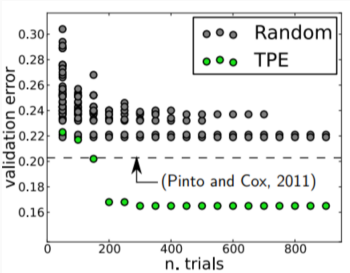


Model-based hyperparameter optimization

Idea:

- Perform a training using a set of hyperparameters.
- Define the cost function to be optimize as the validation set error.
- Use sequential model-based optimization (SMBO) approach, or algorithms which monitors the numerical gradient from the loss function.

Example: Tree-structured Parzen Estimator (TPE)



SMBO

SMBO minimizes functions $f : X \to \mathcal{R}$ where each evaluation is very expensive.

The f function is replaced by a **surrogate** function, $\bar{f}$, easier to manage.

The surrogate function proposes a new search point $\mathbf{x}_{i+1}$, $f(\mathbf{x}_{i+1})$ is computed and $\bar{f}$ updated or recomputed to approximate better the true loss function.

Data: loss function f, initial surrogate $\overline{f}_0$, number of trials T
Result: Candidate $\mathbf{x}_{best}$ for the minimum of f
Set trials history $H = \emptyset$;
for $i = 1$ **to** T **do**
  $x^\star \leftarrow \text{argmin}_{\mathbf{x}} L(\mathbf{x}, \overline{f}_{i-1})$;
  Compute $f(x^\star)$;
  $H \leftarrow H \cup \{\mathbf{x}^\star, f(\mathbf{x}^\star)\}$;
  Model a new surrogate function f_i using H;
end

Where $L(\mathbf{x}, \bar{f})$, the criterion, and $\bar{f}$ depend on the specific algorithm.

TPE

The Tree-structured Parzen Estimator (TPE) algorithm is a SMBO where the surrogate model is a probabilistic model $p(y|\mathbf{x})$, which chooses the next trial point by optimizing the **Expected Improvement** criterion:

$$\mathrm{EI}_{y^\star}(\mathbf{x}) = \int_{-\infty}^{\infty} \max(y^\star - y, 0) p(y|\mathbf{x}) dy$$

which measures how much the loss function is expected to be lower than a threshold value $y^\star$, chosen so that $p(y < y\star) = \gamma$ where γ is a parameter of the algorithm.

TPE

The $p(y|\mathbf{x})$ is computed via Bayes' theorem through $p(\mathbf{x}|y)$

$$p(\mathbf{x}|y) = l(\mathbf{x}) \quad \text{if } y < y^\star; \qquad g(\mathbf{x}) \quad \text{if } y \geq y^\star$$

where $l(\mathbf{x})$ and $g(\mathbf{x})$ are probability distributions estimated by using the trials $\mathbf{x}_i$ such that $f(\mathbf{x}_i)$ is respectively lower and higher or equal $y^\star$.

The Expected Improvement for the TPE admits a closed form solution:

$$\mathrm{EI}_{y^\star}(\mathbf{x}) = \int_{-\infty}^{\infty} \max(y^\star - y) \frac{p(\mathbf{x}|y)p(y)}{p(\mathbf{x})} dy \propto \left(\gamma + \frac{g(\mathbf{x})}{l(\mathbf{x})}(1-\gamma)\right)^{-1}$$

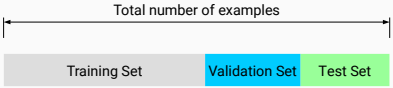
Code libraries and algorithms

- scikit-learn: grid and random search.
- Hyperopt: grid, random and TPE.
- Optuna: grid, random, TPE, CMAES.
- Ray Tune: grid, random, bandit, blended, cost-frugal, TPE, gradient-free, etc.

Cross-validation

Cross-validation

The hyperparameter tune procedure still requires the training/validation/test split to choose for the best model.

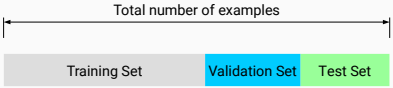


Problems:

- how to perform the data split when the available data set is small?
- how to define a suitable split?

Cross-validation

The hyperparameter tune procedure still requires the training/validation/test split to choose for the best model.



Problems:

- how to perform the data split when the available data set is small?
- how to define a suitable split?

Solution:

Use cross-validation algorithms to access the quality of your model + hyperparameter choice.

Cross-validation

Cross-validation performs a rotation estimation by:

1. **partitioning data** into **training/validation** subsets

Cross-validation

Cross-validation performs a rotation estimation by:

1. **partitioning data** into **training/validation** subsets
2. multiple rounds of cross-validation using different partitions

Cross-validation

Cross-validation performs a rotation estimation by:

1. **partitioning data** into **training/validation** subsets
2. multiple rounds of cross-validation using different partitions
3. results are averaged over the rounds to give an estimate of the model performance

Cross-validation

Cross-validation performs a rotation estimation by:

1. **partitioning data** into **training/validation** subsets
2. multiple rounds of cross-validation using different partitions
3. results are averaged over the rounds to give an estimate of the model performance

Common approaches to cross-validation:

- Exhaustive cross-validation: test all possible ways to divide the original sample into a training and a validation set.

Cross-validation

Cross-validation performs a rotation estimation by:

1. **partitioning data** into **training/validation** subsets
2. multiple rounds of cross-validation using different partitions
3. results are averaged over the rounds to give an estimate of the model performance

Common approaches to cross-validation:

- Exhaustive cross-validation: test all possible ways to divide the original sample into a training and a validation set.
  - Leave-p-out: uses p observations as validation set.

Cross-validation

Cross-validation performs a rotation estimation by:

1. **partitioning data** into **training/validation** subsets
2. multiple rounds of cross-validation using different partitions
3. results are averaged over the rounds to give an estimate of the model performance

Common approaches to cross-validation:

- Exhaustive cross-validation: test all possible ways to divide the original sample into a training and a validation set.
  - Leave-p-out: uses p observations as validation set.
  - Leave-one-out: set $p = 1$.

Cross-validation

Cross-validation performs a rotation estimation by:

1. **partitioning data** into **training/validation** subsets
2. multiple rounds of cross-validation using different partitions
3. results are averaged over the rounds to give an estimate of the model performance

Common approaches to cross-validation:

- Exhaustive cross-validation: test all possible ways to divide the original sample into a training and a validation set.
  - Leave-p-out: uses p observations as validation set.
  - Leave-one-out: set $p = 1$.
- Non-exhaustive cross-validation: do not test all possible ways to divide the original sample but use discrete subsamples.

Cross-validation

Cross-validation performs a rotation estimation by:

1. **partitioning data** into **training/validation** subsets
2. multiple rounds of cross-validation using different partitions
3. results are averaged over the rounds to give an estimate of the model performance

Common approaches to cross-validation:

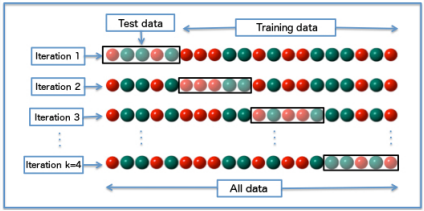
- Exhaustive cross-validation: test all possible ways to divide the original sample into a training and a validation set.
  - Leave-p-out: uses p observations as validation set.
  - Leave-one-out: set $p = 1$.
- Non-exhaustive cross-validation: do not test all possible ways to divide the original sample but use discrete subsamples.
  - k-fold cross-validation.

Example k-fold cross-validation

k-fold cross-validation:

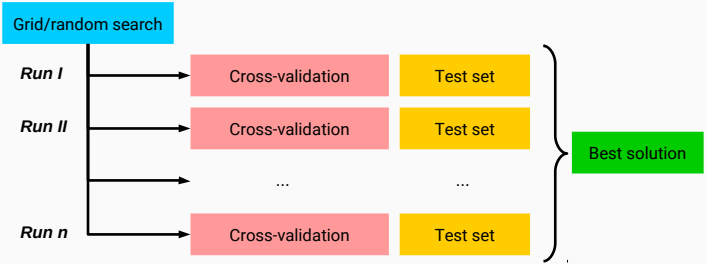
1. the original data is randomly partitioned into k equal sized subsamples.
2. from the k subsamples, a single subsample is used as validation data and the remaining $k-1$ subsamples are used as training data.
3. repeat the process k times by changing the validation and training partitions.
4. compute the average over the k results.

Example of k-fold with $k = 4$:



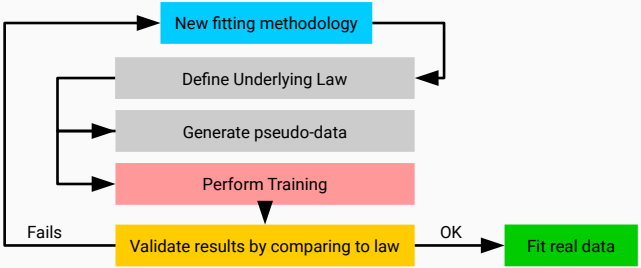
Complete recipe

Perform hyperparameter tune coupled to cross-validation:



Easy parallelization at search and cross-validation stages.

Closure testing

Closure tests

Validation and optimization of fitting strategy performed on **closure test** with known underlying law.